

ADMINISTRACIÓN LINUX

GESTIÓN DE PROCESOS

Fundamentos, comandos y mejores prácticas para el control y monitoreo del sistema.

 Recursos |  Comandos |  Rendimiento

```
0.0%] Task: 34, 60 thr
0.0%] Load average: 0.05
0.7%] Uptime: 00:27:43
0.7%]
561M/15.6G]
0K/0K]
```

HR	S	CPU%	MEM%	TIME+	COMMAND
40	S	0.7	2.4	0:14.00	/usr/sbin/sshd -d
40	S	0.7	2.4	0:00.10	/usr/sbin/sshd -d
56	R	0.7	0.0	0:00.00	htop
28	S	0.0	0.1	0:02.07	/sbin/init
20	S	0.0	0.1	0:00.32	/lib/systemd/systemd
72	S	0.0	0.2	0:00.21	/sbin/multipathd -d
40	S	0.0	0.0	0:00.23	/lib/systemd/systemd
72	S	0.0	0.2	0:00.00	/sbin/multipathd -d
72	S	0.0	0.2	0:00.00	/sbin/multipathd -d
72	S	0.0	0.2	0:00.00	/sbin/multipathd -d
72	S	0.0	0.2	0:00.00	/sbin/multipathd -d
72	S	0.0	0.2	0:00.09	/sbin/multipathd -d
72	S	0.0	0.2	0:00.00	/sbin/multipathd -d
72	S	0.0	0.0	0:00.06	/lib/systemd/systemd
72	S	0.0	0.0	0:00.00	/lib/systemd/systemd
80	S	0.0	0.1	0:00.00	/lib/systemd/systemd
92	S	0.0	0.1	0:00.06	/lib/systemd/systemd
84	S	0.0	0.0	0:00.00	/usr/sbin/cron -f -s
20	S	0.0	0.0	0:00.02	@dbus-daemon --sys
56	S	0.0	0.0	0:00.08	/usr/sbin/irqbalance
04	S	0.0	0.1	0:00.46	/usr/bin/python3 /usr
96	S	0.0	0.0	0:01.09	/usr/sbin/qemu-ga
32	S	0.0	0.0	0:00.01	/usr/sbin/rsyslogd
96	S	0.0	0.0	0:00.00	/usr/sbin/qemu-ga
56	S	0.0	0.0	0:00.00	/usr/sbin/irqbalance

PID

Identificación de Procesos

SEÑALES

Control y Terminación

JOBS

Trabajos en Segundo Plano

| ¿QUÉ ES UN PROCESO?



DEFINICIÓN

Un **proceso** es una instancia de un programa en ejecución. Incluye el código del programa, sus datos actuales, y los recursos del sistema asignados.

Cada proceso tiene un **PID** (Process ID) único y es gestionado por el kernel.



CPU

Tiempo de procesamiento. Determina la velocidad de ejecución.



MEMORIA RAM

Espacio de trabajo temporal. Almacena instrucciones y datos activos.



E/S (I/O)

Lectura/escritura en disco y red. Punto común de cuello de botella.

COMANDOS DE GESTIÓN: PID & PS



PID

Process ID: Identificador único numérico asignado por el kernel a cada proceso en ejecución.

- ✓ **PPID:** PID del proceso padre.
- ✓ **Init:** PID 1 es el ancestro de todos.

COMANDO PS

Reporta una instantánea de los procesos actuales.

Listar todos los procesos:

```
ps aux
```

Formato estándar:

```
ps -ef
```

USER: Dueño **PID:** ID **%CPU:** Uso CPU **%MEM:** Uso RAM

MONITORIZACIÓN EN TIEMPO REAL

TOP

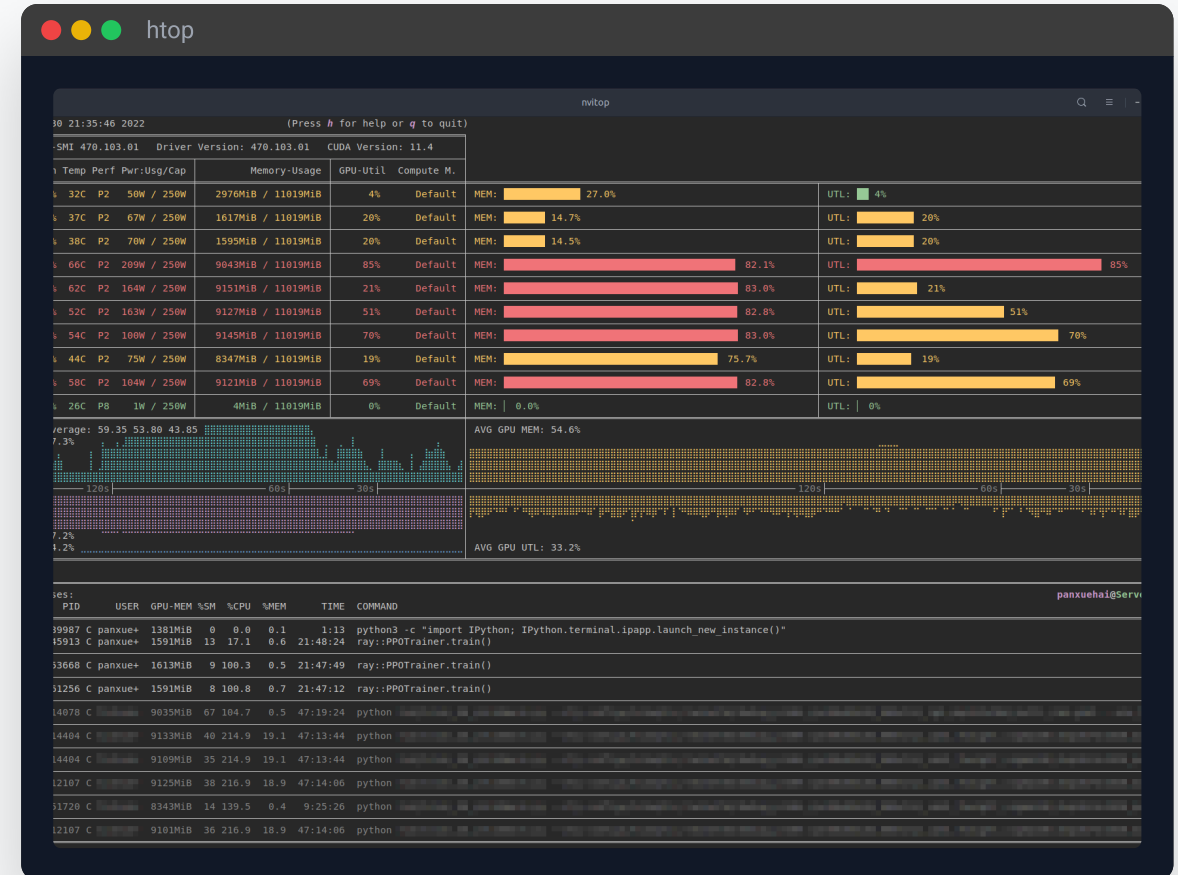
El clásico visor de procesos del sistema.

- ▶ Interfaz básica en terminal
- ▶ Disponible por defecto
- ▶ Atajos de teclado limitados

HTOP

Visor interactivo mejorado.

- ✓ Interfaz gráfica (ncurses)
- ✓ Scroll y selección con ratón
- ✓ Barras visuales de uso
- ✓ Kill y Renice directo



| SEÑALES EN LINUX



¿QUÉ SON?

Las señales son **interrupciones de software** enviadas a un proceso para notificarle de un evento asíncrono.

Pueden ser generadas por:

-  El usuario (Ctrl+C)
-  El kernel (Error de segmento)
-  Otros procesos (comando kill)

≡ SEÑALES COMUNES

SIGTERM (15)

Terminación educada

SIGKILL (9)

Terminación forzada

SIGINT (2)

Interrupción (Ctrl+C)

SIGHUP (1)

Colgar terminal

LISTAR SEÑALES DISPONIBLES

Usa el siguiente comando para ver todas

```
kill -l
```

| COMANDO KILL



FUNCIÓN PRINCIPAL

Envía **señales** a procesos específicos usando su PID. No solo sirve para "matar".

1

PID Obligatorio

Requiere el identificador numérico exacto.

2

Señal por Defecto

Si no se especifica, envía `SIGTERM (15)`.

<> SINTAXIS

```
kill [-s SEÑAL] PID
```

EJEMPLO 1 Terminación Amable

```
$ kill 1234
```

Envía SIGTERM. El proceso puede limpiar y cerrar.

EJEMPLO 2 Forzar Cierre

```
$ kill -9 5678
```

Envía SIGKILL. El kernel termina el proceso inmediatamente.

| JOBS EN LINUX



¿QUÉ ES UN JOB?

Un **job** es un proceso que está **vinculado a una sesión de terminal** específica.

Características principales:

- ✓ Pertenece a una **shell** activa
- ✓ Se identifica con un **número de job** (no PID)
- ✓ Finaliza si se cierra la **terminal**

👁️ ÁMBITO DE APLICACIÓN

Solo son visibles y gestionables desde la **terminal** donde se iniciaron.

```
$ jobs
```

↔️ JOB VS PROCESO

JOB

Local a la Shell

PROCESO

Global al Sistema

COMANDOS DE TRABAJOS

EJECUCIÓN EN BACKGROUND

Añade `&` al final del comando.

```
$ comando_largo &
```

LISTAR TRABAJOS

Muestra los jobs de la sesión actual

```
jobs
```

FOREGROUND

```
fg %1
```

Trae el job al frente

BACKGROUND

```
bg %1
```

Reanuda en segundo plano

SUSPENDER PROCESO

Detiene temporalmente el proceso en foreground.

```
Ctrl + Z
```



Tip: Usa `disown` para desvincular un job de la shell antes de cerrarla.

| VISUALIZACIÓN DE MEMORIA



FREE

Muestra la cantidad de **memoria libre y usada** en el sistema.

```
$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7.7Gi	4.2Gi	1.5Gi	256Mi	2.0Gi	3.0Gi
Swap:	2.0Gi	0.5Gi	1.5Gi			

 Usa **-h** para formato legible (MB/GB).



VMSTAT

Reporta estadísticas de **memoria virtual**, procesos y actividad de CPU.

```
$ vmstat 1 5
```

Muestra 5 informes con intervalo de 1 segundo.



SWAP
Si/So



I/O
Bi/Bo



SYSTEM
In/Cs

| INFORMACIÓN DEL SISTEMA

UPTIME

Muestra cuánto tiempo lleva el **sistema encendido**, usuarios conectados y la **carga media**.

```
$ uptime
```

```
14:23:01 up 10 days, 3:45, 2 users, load average: 0.75,  
0.50, 0.30
```

Interpretación de Load Average:

 1 min  5 min  15 min

W

Muestra quién está conectado y qué están haciendo.

```
$ w
```

WHO

Lista los usuarios conectados al sistema.

```
$ who
```

DF

Espacio en disco disponible en los sistemas de archivos.

```
$ df -h
```

EJERCICIOS PRÁCTICOS: SLEEP & &



LABORATORIO

Usa `sleep` para simular procesos largos y `&` para ejecutar en segundo plano.

Flujo de práctica:

Lanzar proceso en background

Verificar con `jobs`

Manipular (`fg/bg`)

Finalizar con `kill`

1 INICIAR PROCESO

```
$ sleep 300 &
```

Espera 5 minutos en segundo plano. Devuelve [jobnum] PID.

2 VERIFICAR Y TRAER

```
$ jobs
```

```
$ fg %1
```

3 TERMINAR PROCESO

```
$ kill %1
```

Usa % para referenciar el Job ID, no el PID.

CONCLUSIONES



CONCEPTOS CLAVE

- ✓ **Procesos:** Unidades de ejecución con consumo de recursos monitorizable.
- ✓ **Señales:** Mecanismo de comunicación asíncrono para controlar procesos.
- ✓ **Jobs:** Procesos ligados a la sesión de terminal actual.
- ✓ **Monitorización:** Uso de comandos específicos para RAM y estado del sistema.



BUENAS PRÁCTICAS

- Usa `kill -15` antes que `-9`
- Verifica el PID antes de matar un proceso
- Monitoriza recursos regularmente



COMANDOS ESENCIALES

`ps, top, htop`

`kill, kill -l`

`jobs, fg, bg`

`free, vmstat`